

Université Sorbonne Paris Nord
Ecole d'Ingénieurs Sup Galilée

TP2

Réalisé par:

BRAHIMI Kaoutar

Exercise 1 :

1-

```
SELECT dept_name
FROM department
WHERE budget IN ( SELECT MAX(budget) FROM department );
```

The screenshot shows a SQL query execution interface. The query is: `SELECT dept_name FROM department WHERE budget IN ( SELECT MAX(budget) FROM department );`. The results are displayed in a table with one column, `DEPT_NAME`, and one row, `Finance`. The interface includes tabs for `Results`, `Explain`, `Describe`, `Saved SQL`, and `History`. At the bottom, it indicates `1 rows returned in 0.00 seconds` and provides a `Download` link.

DEPT_NAME
Finance

2-

```
SELECT T.name, T.salary
FROM teacher T
WHERE T.salary > (
    SELECT AVG(B.salary)
    FROM teacher B
);
```

The screenshot shows a SQL query execution interface. The query is: `SELECT T.name, T.salary FROM teacher T WHERE T.salary > ( SELECT AVG(B.salary) FROM teacher B );`. The results are displayed in a table with two columns, `NAME` and `SALARY`, and seven rows: `Einstein` (95000), `Gold` (87000), `Katz` (75000), `Singh` (80000), `Brandt` (92000), and `Kim` (80000). The interface includes tabs for `Results`, `Explain`, `Describe`, `Saved SQL`, and `History`. At the bottom, it indicates `7 rows returned in 0.01 seconds` and provides a `Download` link.

NAME	SALARY
Einstein	95000
Gold	87000
Katz	75000
Singh	80000
Brandt	92000
Kim	80000

3- SELECT teacher.name AS teacher_name, student.name AS student_name, COUNT(*) AS common_courses

```
FROM teacher
JOIN teaches ON teacher.ID = teaches.ID
JOIN takes ON takes.course_id = teaches.course_id
    AND takes.sec_id = teaches.sec_id
    AND takes.semester = teaches.semester
    AND takes.year = teaches.year
JOIN student ON student.ID = takes.ID
GROUP BY teacher.name, student.name
HAVING COUNT(*) >= 2;
```

```

1 SELECT teacher.name AS teacher_name, student.name AS student_name, COUNT(*) AS common_courses
2 FROM teacher
3 JOIN teaches ON teacher.ID = teaches.ID

```

TEACHER_NAME	STUDENT_NAME	COMMON_COURSES
Srinivasan	Bourikas	2
Srinivasan	Shankar	3
Crick	Tanaka	2
Katz	Levy	2
Srinivasan	Zhang	2

5 rows returned in 0.05 seconds [Download](#)

4-

```

SELECT T.teacher_name, T.student_name, T.total_count
FROM (
    SELECT teacher.name AS teacher_name, student.name AS student_name, COUNT(*) AS
total_count
    FROM teacher
    JOIN teaches ON teacher.ID = teaches.ID
    JOIN takes ON takes.course_id = teaches.course_id
        AND takes.sec_id = teaches.sec_id
        AND takes.semester = teaches.semester
        AND takes.year = teaches.year
    JOIN student ON student.ID = takes.ID
    GROUP BY teacher.name, student.name
) T
WHERE T.total_count >= 2
ORDER BY T.teacher_name;

```

```

1 SELECT T.teacher_name, T.student_name, T.total_count
2 FROM (
3     SELECT teacher.name AS teacher_name, student.name AS student_name, COUNT(*) AS total_count

```

TEACHER_NAME	STUDENT_NAME	TOTAL_COUNT
Crick	Tanaka	2
Katz	Levy	2
Srinivasan	Bourikas	2
Srinivasan	Shankar	3
Srinivasan	Zhang	2

5 rows returned in 0.04 seconds [Download](#)

5-

```

SELECT ID, name
FROM student
WHERE ID NOT IN (
    SELECT student.ID

```

```

FROM student
JOIN takes ON takes.ID = student.ID
WHERE takes.year < 2009
);

```

```

1  SELECT ID, name
2  FROM student
3  WHERE ID NOT IN (

```

ID	NAME
76653	Aoi
23121	Chavez
55739	Sanchez
00128	Zhang
98765	Bourikas

More than 10 rows available. Increase rows selector to view more rows.

10 rows returned in 0.02 seconds [Download](#)

6-

```

SELECT *
FROM teacher
WHERE name LIKE 'E%';

```

```

1  SELECT *
2  FROM teacher
3  WHERE name LIKE 'E%';

```

ID	NAME	DEPT_NAME	SALARY
22222	Einstein	Physics	95000
32343	El Said	History	60000

2 rows returned in 0.00 seconds [Download](#)

7-

```

SELECT name
FROM teacher T1
WHERE 3 = (
    SELECT COUNT(DISTINCT T2.salary)
    FROM teacher T2
    WHERE T2.salary > T1.salary
);

```

```

1  SELECT name
2  FROM teacher T1
3  WHERE 3 = (

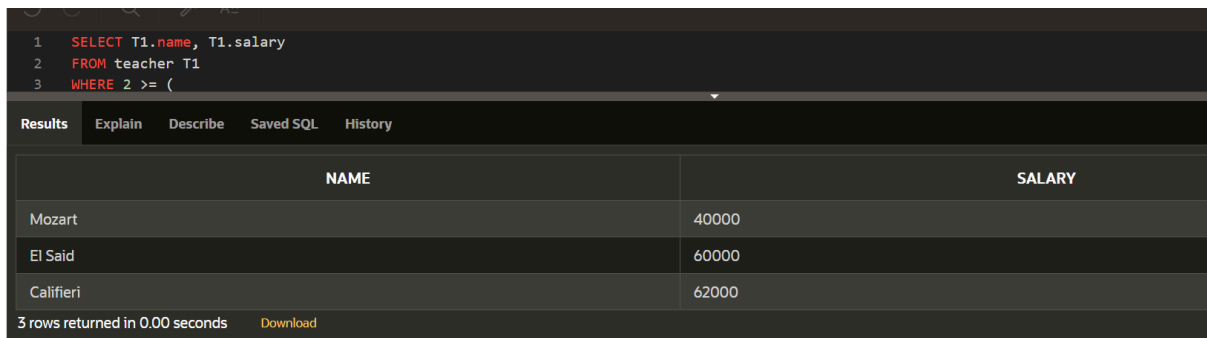
```

NAME
Gold

1 rows returned in 0.01 seconds [Download](#)

8-

```
SELECT T1.name, T1.salary
FROM teacher T1
WHERE 2 >= (
  SELECT COUNT(DISTINCT T2.salary)
  FROM teacher T2
  WHERE T2.salary < T1.salary
)
ORDER BY T1.salary ASC;
```



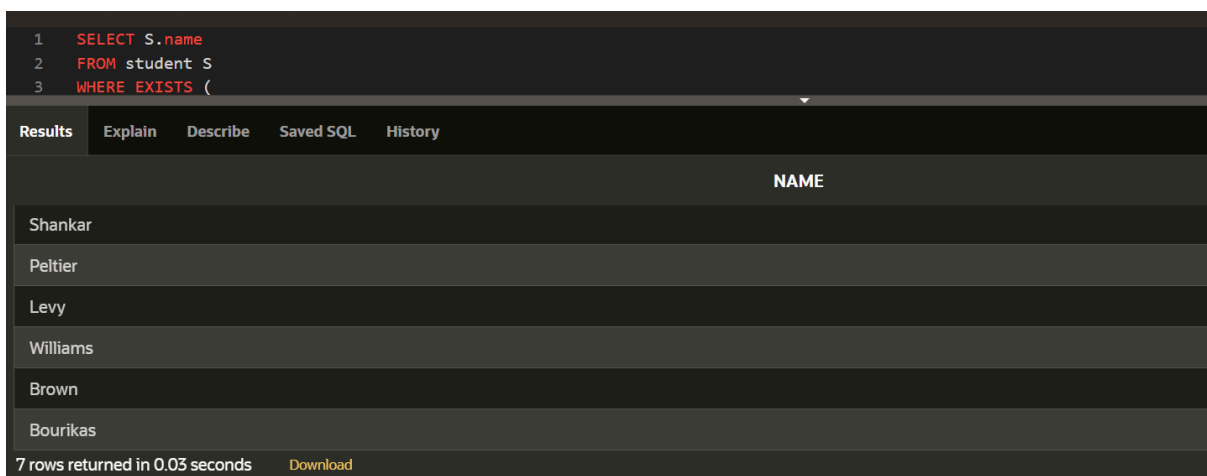
The screenshot shows a SQL query execution interface. The query is:
1 SELECT T1.name, T1.salary
2 FROM teacher T1
3 WHERE 2 >= (
 SELECT COUNT(DISTINCT T2.salary)
 FROM teacher T2
 WHERE T2.salary < T1.salary
)
ORDER BY T1.salary ASC;
The interface has tabs for Results, Explain, Describe, Saved SQL, and History. The Results tab is active, showing a table with two columns: NAME and SALARY. The table contains three rows: Mozart with salary 40000, El Said with salary 60000, and Califieri with salary 62000. At the bottom, it says '3 rows returned in 0.00 seconds' and has a 'Download' link.

NAME	SALARY
Mozart	40000
El Said	60000
Califieri	62000

3 rows returned in 0.00 seconds [Download](#)

9-

```
SELECT S.name
FROM student S
WHERE EXISTS (
  SELECT 1
  FROM takes
  WHERE takes.ID = S.ID
  AND takes.semester = 'Fall'
  AND takes.year = 2009
);
```



The screenshot shows a SQL query execution interface. The query is:
1 SELECT S.name
2 FROM student S
3 WHERE EXISTS (
 SELECT 1
 FROM takes
 WHERE takes.ID = S.ID
 AND takes.semester = 'Fall'
 AND takes.year = 2009
)
The interface has tabs for Results, Explain, Describe, Saved SQL, and History. The Results tab is active, showing a table with one column: NAME. The table contains seven rows: Shankar, Peltier, Levy, Williams, Brown, and Bourikas. At the bottom, it says '7 rows returned in 0.03 seconds' and has a 'Download' link.

NAME
Shankar
Peltier
Levy
Williams
Brown
Bourikas

7 rows returned in 0.03 seconds [Download](#)

10-

```
SELECT S.name
```

```

FROM student S
  WHERE EXISTS (
    SELECT 1
    FROM takes
    WHERE takes.ID = S.ID
      AND takes.semester = 'Fall'
      AND takes.year = 2009
  );

```

1	SELECT S.name
2	FROM student S
3	WHERE EXISTS (

Results	Explain	Describe	Saved SQL	History
NAME				
Zhang				
Shankar				
Peltier				
Levy				
Williams				
Brown				

11-

```

SELECT DISTINCT student.name
FROM student
JOIN takes ON student.ID = takes.ID
WHERE takes.semester = 'Fall' AND takes.year = 2009;

```

1	SELECT DISTINCT student.name
2	FROM student
3	JOIN takes ON student.ID = takes.ID

Results	Explain	Describe	Saved SQL	History
NAME				
Zhang				
Levy				
Bourikas				
Shankar				
Peltier				
Williams				

7 rows returned in 0.05 seconds [Download](#)

12-

```

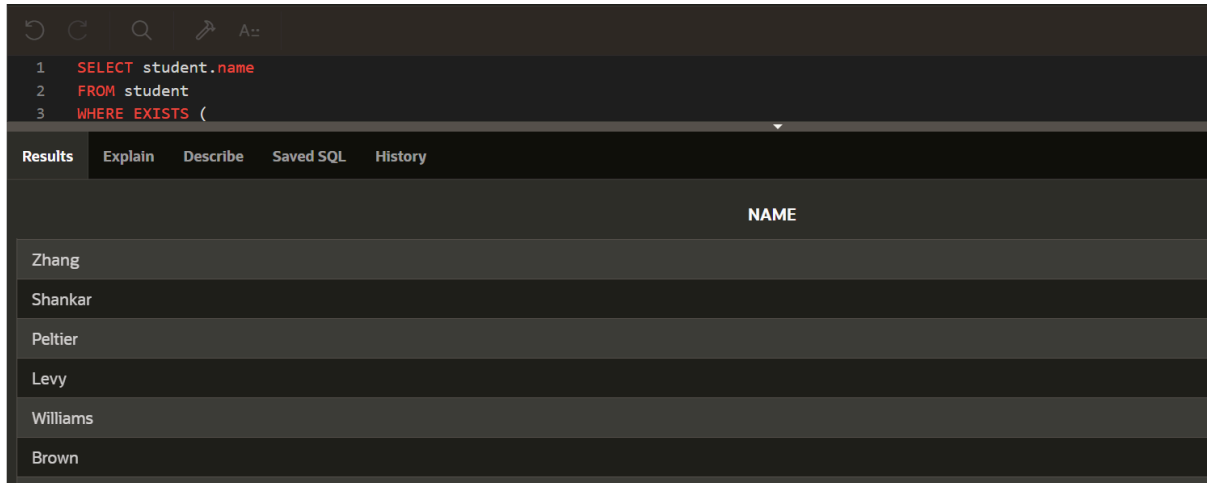
SELECT student.name
FROM student
WHERE EXISTS (
  SELECT 1
  FROM takes

```

```

WHERE takes.ID = student.ID
AND takes.semester = 'Fall'
AND takes.year = 2009
);

```



The screenshot shows a SQL query editor with the following query:

```

1 SELECT student.name
2 FROM student
3 WHERE EXISTS (

```

The results tab is active, showing a table with one column named 'NAME' and six rows of student names:

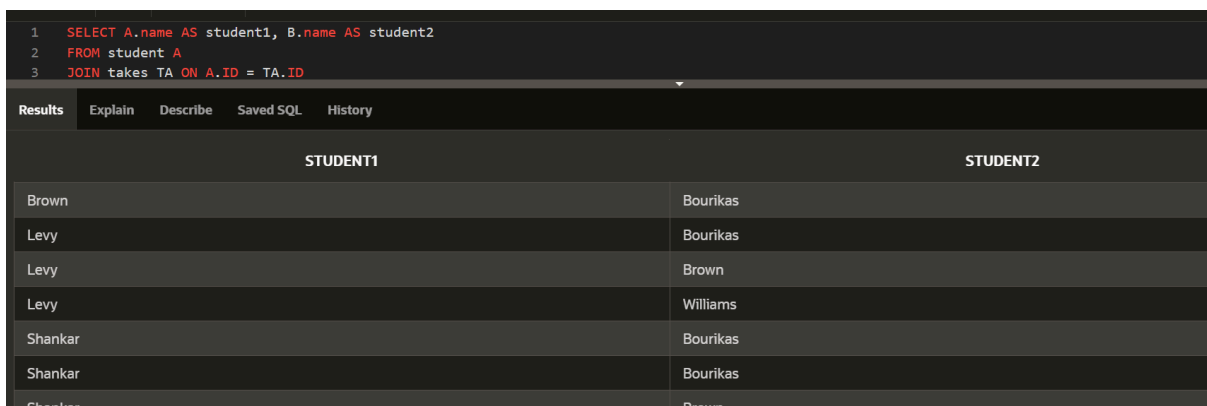
NAME
Zhang
Shankar
Peltier
Levy
Williams
Brown

13-

```

SELECT A.name AS student1, B.name AS student2
FROM student A
JOIN takes TA ON A.ID = TA.ID
JOIN takes TB ON TA.course_id = TB.course_id
AND TA.sec_id = TB.sec_id
AND TA.semester = TB.semester
AND TA.year = TB.year
JOIN student B ON B.ID = TB.ID
WHERE A.ID < B.ID
ORDER BY A.name, B.name;

```



The screenshot shows a SQL query editor with the following query:

```

1 SELECT A.name AS student1, B.name AS student2
2 FROM student A
3 JOIN takes TA ON A.ID = TA.ID

```

The results tab is active, showing a table with two columns: 'STUDENT1' and 'STUDENT2'. The results are as follows:

STUDENT1	STUDENT2
Brown	Bourikas
Levy	Bourikas
Levy	Brown
Levy	Williams
Shankar	Bourikas
Shankar	Bourikas
Shankar	Brown

Exercise 2 :

Cas 1 : $R(A, B, C)$ avec $F = \{A \rightarrow B, B \rightarrow C\}$

Dans cette relation, on observe que A détermine B, et B détermine C, ce qui signifie par transitivité que A détermine aussi C. Ainsi, A est une clé candidate, et même la clé primaire.

La relation est en 1NF car chaque champ contient des valeurs indivisibles.

Elle est également en 2NF, étant donné que les attributs non clés (B et C) dépendent entièrement de la clé primaire A.

Cependant, elle n'est pas en 3NF en raison de la dépendance transitive $A \rightarrow B \rightarrow C$. Cela constitue une violation de la 3NF.

De plus, la BCNF n'est pas respectée, car la dépendance $B \rightarrow C$ ne provient pas d'une super-clé.

Pour normaliser cette relation en 3NF, il faut la décomposer en deux relations :

- **$R1(A, B)$**
- **$R2(B, C)$**
Cette décomposition élimine la dépendance transitive tout en conservant les dépendances fonctionnelles essentielles.

Cas 2 : $R(A, B, C)$ avec $F = \{A \rightarrow C, A \rightarrow B\}$

Dans ce cas, A détermine à la fois B et C, ce qui en fait la clé primaire de la relation.

La relation est en 1NF car les données sont atomiques.

Elle respecte également la 2NF car chaque attribut dépend totalement de la clé primaire (A).

Aucune dépendance transitive n'existe, donc la 3NF est aussi satisfaite.

Enfin, puisque toutes les dépendances fonctionnelles sont basées sur une super-clé (A), la relation est aussi conforme à la BCNF.

Il n'est donc pas nécessaire de la décomposer.

Cas 3 : $R(A, B, C)$ avec $F = \{A, B \rightarrow C ; C \rightarrow B\}$

Ici, la combinaison (A, B) détermine C, ce qui en fait la clé primaire. En parallèle, C détermine B, ce qui introduit une dépendance fonctionnelle entre un attribut non clé (C) et un composant de la clé (B).

Cela indique une violation de la 2NF, car B dépend d'une partie seulement de la clé (via C), et donc la 3NF et la BCNF ne sont pas respectées non plus.

Pour corriger cette anomalie et atteindre la 2NF, on peut scinder la relation en deux sous-relations :

- **$R1(C, B)$** , qui capte la dépendance $C \rightarrow B$
- **$R2(A, C)$** , qui conserve la dépendance $A, B \rightarrow C$ sous la forme implicite $A \rightarrow C$ avec la dépendance $C \rightarrow B$ conservée dans R1.